

Projet — Développement d'Applications Internet

Licence 2 - Université de Pau et des Pays de l'Adour - Collège STEE
2025-2026

1. Contexte

Les plateformes immobilières en ligne (SeLoger, Leboncoin, etc.) sont devenues incontournables pour trouver un logement à louer, à acheter ou pour réserver une location saisonnière. Elles proposent généralement :

- des **filtres** (type de transaction, région, type de bien, prix, etc.) ;
- des **listes de résultats** avec pagination et tri ;
- des **fiches détaillées** pour chaque bien ;
- un système de **favoris** ;
- un formulaire de **contact** ou de demande de visite ;
- parfois la possibilité d'**ajouter, modifier ou supprimer** une annonce.

Le site que vous allez créer est une version simplifiée de ce type d'application, entièrement en HTML/CSS/JavaScript côté client. L'application doit être responsive.

Résumé fonctionnel. L'application permet de :

- rechercher des biens immobiliers (location, vente, saisonnier) ;
- filtrer les annonces selon plusieurs critères (région, type de bien, prix, favoris) ;
- trier les résultats (prix / date) ;
- afficher les résultats avec **pagination** (6 annonces par page) ;
- gérer une liste de **favoris** en mémoire (tableau JavaScript) ;
- consulter une **fiche détaillée** avec un **carrousel de photos** ;
- **ajouter une nouvelle annonce** via un formulaire dédié ;
- **modifier ou supprimer** une annonce existante ;
- envoyer une **demande de contact** via un formulaire validé en JavaScript.

L'application doit fonctionner comme une *single-page application* simple : aucune page ne doit être rechargée.

2. Description générale de l'application

2.1. Structure générale

Votre application comportera :

- un en-tête avec :

- le logo texte du site ;
- un menu de navigation interne avec au moins deux liens :
 - **Accueil** : amène à la section de recherche / résultats ;
 - **Contact** : amène à la section du formulaire de contact.
- un corps de page avec plusieurs sections :
 1. **Recherche rapide** : formulaire de recherche et filtre favoris ;
 2. **Résultats** : liste paginée des annonces + tri + accès aux détails ;
 3. **Ajout / modification d'annonce** : formulaire permettant de créer ou éditer une annonce ;
 4. **Détail** : fiche détaillée d'une annonce avec carrousel de photos (affichée au clic sur Détails) ;
 5. **Contact** : formulaire de contact avec validation.
- un pied de page simple, par exemple : ImmoFR © 2025.

La navigation Accueil/Contact se fait par des ancres au sein de la même page HTML.

2.2. Organisation des fichiers

On recommande l'organisation suivante :

- `index.html` : structure HTML de la page ;
- `styles.css` : feuilles de style ;
- `data.js` : données d'exemple (annonces) sous forme de tableau d'objets ;
- `app.js` : logique JavaScript de l'application (DOM, événements, filtrage, tri, pagination, ajout/modification/suppression, etc.) ;
- `images/` : répertoire contenant les photos utilisées.

Les données seront codées en dur dans `data.js`.

3. Fonctionnalités à implémenter

3.1. Section « Recherche rapide »

Cette section contient un **formulaire HTML** de recherche (élément `<form>`) avec les champs suivants :

- Type de transaction (**Location**, **Vente**, **Saisonnier**, ou « Tous ») ;
- Région (au minimum quelques régions françaises, plus « Toute la France ») ;
- Type de bien (**Appartement**, **Studio**, **Maison**, **Local commercial**, **Terrain** ou « Tous ») ;
- Prix min (€, champ numérique optionnel) ;
- Prix max (€, champ numérique optionnel) ;
- un bouton **Rechercher**.

Important : Dès qu'il y a un bouton qui déclenche une action à partir de plusieurs champs saisis, vous devez encapsuler ces champs dans un `<form>` et gérer l'événement `submit` en JavaScript (et non pas uniquement des `<input>` isolés).

Sous le formulaire, ajouter une case à cocher :

`Afficher uniquement mes favoris`

qui permettra de filtrer les annonces marquées comme favorites.

Comportement attendu.

- À la soumission du formulaire, la page ne doit pas être rechargée ;
- les valeurs saisies sont récupérées en JavaScript ;
- les annonces affichées dans la section « Résultats » sont filtrées en fonction des critères ;
- le filtre « afficher uniquement mes favoris » restreint la liste aux annonces marquées en favoris dans votre liste en mémoire.

3.2. Section « Résultats » et tri

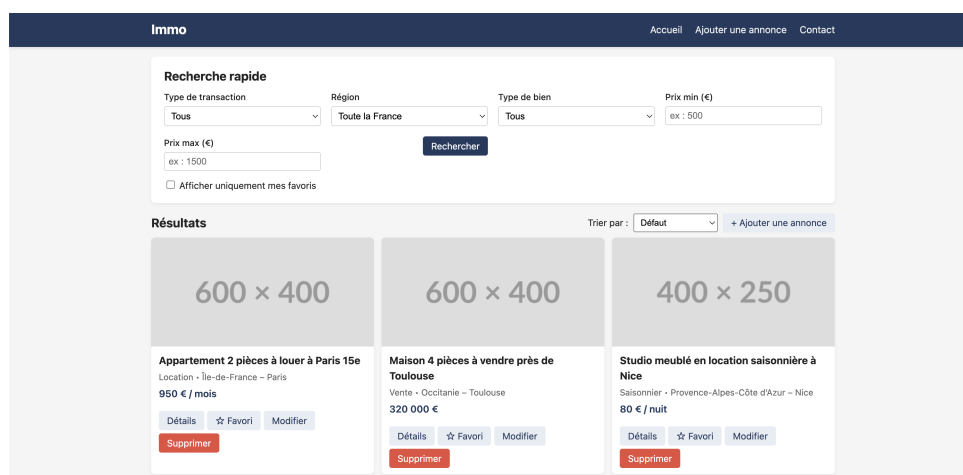
La section Résultats comporte :

- une en-tête de résultats sur une seule ligne contenant :
 - le titre « Résultats » ;
 - un sélecteur de tri placé à droite :

Trier par : [Défaut | Prix croissant | Prix décroissant | Plus récent | Plus ancien]
- une grille de cartes d'annonces générées en JavaScript ;
- une zone de pagination ;
- un message :

Aucun bien ne correspond à votre recherche.

lorsque la liste est vide.



Sections recherche, résultats et tri.

Données des annonces. Les données seront stockées dans `data.js` sous la forme :

```
const annonces = [
  {
    id: 1,
    titre: "Appartement 2 pièces à louer à Paris 15e",
    transaction: "Location",
    region: "Île-de-France",
    ville: "Paris",
    typeBien: "Appartement",
    prix: 950,
    unitePrix: "€ / mois",
    surface: 40,
    pieces: 2,
    adresse: "45 rue XXXX, 64000 Pau",
    image: "images/appartement_paris.jpg",
    images: [
      "images/appartement_paris.jpg",
      "images/appartement_paris_2.jpg"
    ],
    description: "Appartement lumineux...",
    dateAjout: "2025-02-10"
  },
  // ...
];
```

Chaque annonce doit au minimum contenir : `id`, `titre`, `transaction`, `region`, `ville`, `typeBien`, `prix`, `unitePrix`, `surface`, `pieces`, `image` (image principale), `images` (tableau de photos pour le carrousel), `description`, `dateAjout`.

Affichage d'une carte d'annonce. Chaque carte contiendra :

- une image du bien (photo principale);
- le titre de l'annonce;
- une ligne de métadonnées : **Transaction • Région - Ville**;
- le prix formaté (ex. 950 € / mois, 320 000 €, etc.);
- quatre boutons :
 - **Détails** : ouvre la fiche détaillée;
 - **Favori** : ajoute/enlève l'annonce de la liste des favoris;
 - **Modifier** : ouvre le formulaire d'édition pré-rempli;
 - **Supprimer** : supprime l'annonce de la liste (après confirmation).

Tri. Le sélecteur de tri doit appliquer les règles suivantes :

- **Défaut** : l'ordre d'origine du tableau `annonces` est conservé;
- **Prix croissant** : tri par prix (du plus petit au plus grand);

- Prix décroissant : tri par prix (du plus grand au plus petit) ;
- Plus récent : tri par `dateAjout` (annonces les plus récentes en premier) ;
- Plus ancien : tri par `dateAjout` (annonces les plus anciennes en premier).

Le tri s'applique après filtrage et avant la pagination.

3.3. Pagination

La liste filtrée et triée peut contenir un nombre quelconque d'annonces. Vous devez :

- afficher 6 annonces par page ;
- créer une zone de pagination contenant :
 - un bouton « précédent » (⏪) ;
 - les numéros de pages 1 2 3 ... ;
 - un bouton « suivant » (⏩).
- gérer les clics sur les boutons sans rechargement de page.



Résultats et pagination.

La pagination doit toujours être cohérente avec les filtres et le tri :

- lorsqu'on modifie les filtres ou le tri, on revient à la page 1 ;
- les numéros de page s'adaptent au nombre de résultats.

3.4. Gestion des favoris (liste en mémoire)



L'utilisateur a deux favoris.

Chaque carte d'annonce propose un bouton favoris :

- Les favoris peuvent être gérés via les données ou via une **liste d'identifiants** stockée en mémoire (par exemple un tableau `favorisIds` en JavaScript) ;
- Le filtre « Afficher uniquement mes favoris » n'affiche que les annonces dont l'identifiant est présent dans ce tableau ;

- Aucune persistance n'est imposée : par défaut, les favoris sont perdus lorsque la page est rechargée (une extension possible est de les stocker dans `localStorage`, voir section Évaluation).

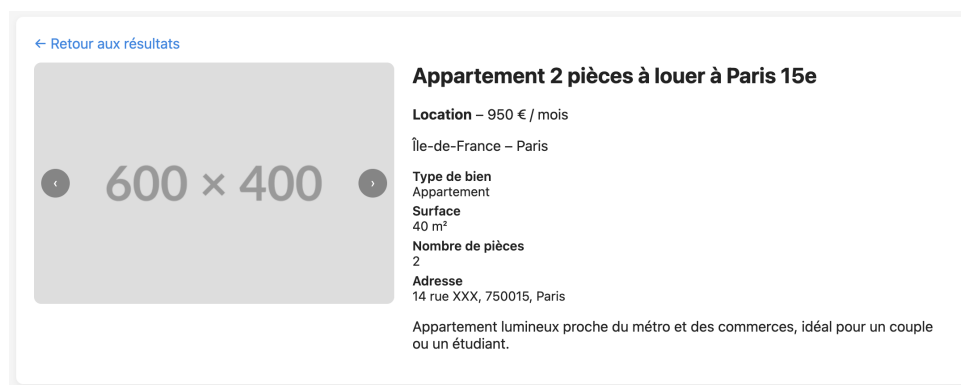
3.5. Fiche détaillée avec carrousel

Lorsque l'utilisateur clique sur **Détails** :

- la section **Résultats** est **masquée** ;
- la section **Détail** est **affichée**.

La fiche détaillée doit comporter :

- un **carrousel d'images** permettant de faire défiler plusieurs photos du bien :
 - le changement de photo se fait en JavaScript, sans rechargement de page ;
 - le carrousel peut « boucler » (après la dernière image, revenir à la première).
- le titre du bien ;
- la transaction et le prix ;
- la région et la ville ;
- une liste d'informations :
 - type de bien ;
 - surface (en m²) ;
 - nombre de pièces ;
 - adresse.
- la description textuelle du bien.



Fiche détaillée.

Un bouton **← Retour aux résultats** permet de revenir à la liste, à la même page de pagination qu'auparavant.

3.6. Ajout, modification et suppression d'une annonce

L'utilisateur doit pouvoir :

- **ajouter** une nouvelle annonce via un formulaire dédié ;
- **modifier** une annonce existante à partir d'une carte (bouton **Modifier**) ;

- **supprimer** une annonce existante (bouton **Supprimer** sur la carte).

Formulaire d'ajout / modification. Une section spécifique « Ajouter une annonce » (ou « Modifier une annonce ») doit contenir un **formulaire HTML** (`<form>`) avec, au minimum, les champs suivants :

- Titre (obligatoire) ;
- Type de transaction (obligatoire) ;
- Région (obligatoire) ;
- Ville (obligatoire) ;
- Type de bien (obligatoire) ;
- Prix (obligatoire, champ numérique) ;
- **Photo principale** (champ `<input type="file">`, optionnel mais recommandé) ;
- **Autres photos** (champ `<input type="file" multiple>`, optionnel) ;
- Description (zone de texte, optionnelle mais souhaitable).
Le formulaire doit proposer au moins deux boutons :
 - **Valider l'annonce** : crée ou met à jour l'annonce en mémoire ;
 - **Annuler** : annule l'opération, réinitialise le formulaire et masque la section.

Important :

- Le formulaire d'ajout / modification doit impérativement être un élément `<form>` HTML.
- La soumission doit être gérée en JavaScript (sans rechargement de page).
- Les nouvelles annonces sont ajoutées dans le tableau **annonces** en mémoire (aucun envoi vers un serveur n'est requis).
- Les images sélectionnées peuvent être gérées côté client via `URL.createObjectURL` ou similaires ; elles ne sont pas réellement déposées sur un serveur.

Modification et suppression.

- Le bouton **Modifier** sur une carte ouvre le formulaire d'édition, pré-rempli avec les informations de l'annonce correspondante.
- Le bouton **Supprimer** sur une carte retire l'annonce de la liste (après une confirmation).
- Après ajout ou modification réussie d'une annonce, un message de confirmation doit apparaître à l'écran (par exemple en bas de la fenêtre) pendant environ 3 secondes, puis disparaître automatiquement. Ce message peut indiquer par exemple « Annonce ajoutée avec succès » ou « Annonce mise à jour avec succès ».
- Après ajout, modification ou suppression, la liste des résultats, le tri, la pagination et le filtrage (dont les favoris) doivent rester cohérents.

3.7. Formulaire de contact

La section **Contact** doit contenir un **formulaire HTML** (élément `<form>`) avec les champs suivants :

- Nom complet (obligatoire) ;
- Email (obligatoire, **format email**) ;
- Type de demande (liste déroulante, obligatoire) :
 - Demande d'information ;
 - Demande de visite ;
 - Proposition d'achat ;
 - Autre.
- **Message** (zone de texte, obligatoire).

The screenshot shows a web form titled "Contact". It contains four main input fields: "Nom complet", "Email", "Type de demande", and "Message". Below the "Nom complet" and "Email" fields, there is a red error message: "Le nom est obligatoire." and "L'email est obligatoire." respectively. Below the "Type de demande" field, there is a dropdown menu with the text "Sélectionnez...". Below the "Message" field, there is a red error message: "Le message est obligatoire." At the bottom of the form, there is a blue button labeled "Envoyer". The footer of the page reads "Immo © 2025".

Formulaire avec un message d'erreur et le pied de page.

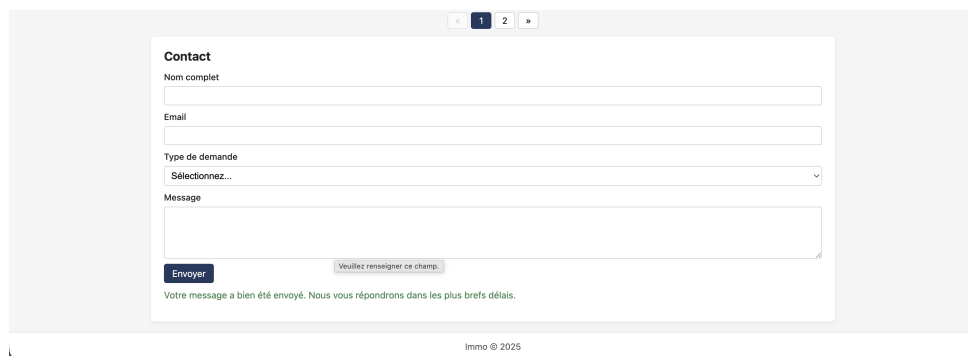
Validation côté client.

- La soumission du formulaire ne doit pas recharger la page.
- Tous les champs sont obligatoires.
- L'email doit respecter un format basique valide.
- En cas d'erreur, un **message d'erreur** est affiché sous le champ concerné et le champ est visuellement mis en évidence (bordure rouge, par exemple).
- Si tous les champs sont valides :
 - les champs sont réinitialisés ;
 - un message de **confirmation** (succès) est affiché dans la page (par exemple en vert).
 - ce message de confirmation doit être **temporaire** : il apparaît pendant environ 3 secondes, puis disparaît automatiquement (par exemple sous la forme d'un petit « toast » en bas de l'écran).

4. Options techniques

4.1. Utilisation de Bootstrap (optionnelle)

Vous pouvez utiliser le framework CSS **Bootstrap** pour simplifier la mise en forme (grilles, boutons, cartes, formulaires, etc.).



Formulaire bien rempli avec un message de confirmation (succès).

4.2. Architecture SAM (State–Action–Model, optionnelle)

L'utilisation de l'architecture **SAM** est fortement encouragée pour structurer le JavaScript.

Rappel SAM.

- **State** : décrit l'état courant de l'application (annonces, filtres, tri, page courante, favoris, annonce détaillée, état du formulaire, etc.) ;
- **Actions** : représentent les intentions de l'utilisateur (filtrer, trier, changer de page, ouvrir un détail, basculer un favori, soumettre un formulaire, ajouter/modifier/supprimer une annonce, etc.) ;
- **Model** : applique la logique métier, met à jour l'état en fonction des actions, puis appelle la **Vue** ;
- **View** : génère le DOM à partir de l'état (rendu de la liste, de la pagination, du détail, des formulaires, des messages d'erreur, etc.).

L'idée centrale est de **séparer** clairement la logique métier du rendu graphique.

5. Contraintes techniques

- Une seule page HTML (`index.html`), navigation interne par ancres ;
- **Interdit** : frameworks JavaScript type React, Vue, Angular, etc. ;
- **Autorisé** : Bootstrap (CSS) ou un autre framework CSS, architecture SAM (JS).
- Tous les formulaires (recherche, ajout/modification, contact) doivent être implémentés avec un élément `<form>` et gérés côté client en JavaScript (pas de rechargement de page) ;
- Code découpé en fonctions claires et réutilisables (par ex. `renderResults()`, `sortList()`, `validateContactForm()`, etc.) ;
- Commenter les fonctions importantes.

6. Livrables

En plus des fichiers du projet, vous ajouterez un fichier `README.md` contenant :

- une brève description de l'application ;
- les fonctionnalités implémentées ;

- les choix techniques (Bootstrap utilisé ou non, SAM utilisé ou non, etc.) ;
- les éventuelles limitations connues.

7. Critères d'évaluation (indicatifs)

- **Fonctionnalités (40 %) :**
 - filtrage correct (transaction, région, type de bien, prix, favoris) ;
 - tri (prix croissant/décroissant, plus récent/plus ancien) ;
 - pagination (6 résultats par page) ;
 - gestion des favoris (via une liste en mémoire) ;
 - fiche détaillée fonctionnelle avec carrousel ;
 - formulaire de contact avec validation ;
 - ajout, modification et suppression d'annonce.
- **Qualité du code JavaScript (30 %) :**
 - lisibilité, indentation, commentaires ;
 - noms de variables et fonctions explicites ;
 - séparation claire entre logique, rendu, utilitaires ;
 - usage pertinent de l'architecture SAM.
- **Qualité de l'interface (20 %) :**
 - mise en page correcte (desktop et petits écrans) ;
 - lisibilité des cartes d'annonces ;
 - messages d'erreur/succès explicites ;
 - formulaires clairement identifiés.
- **Aspect soigné et extensions (10 %) :**
 - proposer un mode sombre (dark mode) ;
 - ajouter un résumé au-dessus des résultats : « X annonces trouvées » ;
 - extension vers une persistance réelle des favoris (par exemple via `localStorage`) ;
 - petits détails ergonomiques (hover, transitions douces, messages de confirmation) ;
 - ajouter un tri par surface ;
 - fonctionnalités supplémentaires pertinentes.
- Utilisation de Git et démarche de développement
 - **dépôt créé dès le début du projet ;**
 - commits réguliers tout au long du développement (et non un unique gros commit à la fin) ;
 - messages de commit explicites ;
 - historique cohérent avec la progression du projet.

8. Procédure de soumission du travail réalisé

Le travail est à soumettre pour le **dimanche 11 janvier 2026 à 23 :59** au plus tard. Il peut être réalisé seul ou en binôme.

Le projet doit **obligatoirement** être rendu via Git (GitLab de l'Université de Pau et des Pays de l'Adour : `git.univ-pau.fr`).

- Créez votre dépôt Git **dès le début du projet** et travaillez dessus tout au long du développement.
- Assurez-vous que votre dépôt Git est **public** afin que l'enseignant puisse y accéder avec le lien.
- Vous enverrez par mail, à l'adresse `issa-moussa.diop@univ-pau.fr`, le lien du dépôt avant la date et l'heure limite.
- **Attention** : n'envoyez pas le projet sous forme d'archive ZIP/RAR en pièce jointe de mail !
- Le mail devra contenir :
 1. les noms et prénoms des membres du groupe ;
 2. adresses mail des membres du groupe
 3. le lien du dépôt Git.
- Tout projet qui n'est pas soumis via Git **ne sera pas accepté**. Des pénalités de retard seront appliquées jusqu'à réception du lien Git (**-2 points par jour**).
- **Attention** : il est de votre responsabilité de vérifier que votre application fonctionne parfaitement une fois le projet déposé dans le dépôt Git !

Un point **bonus** sera attribué aux projets dont les membres ont activement utilisé Git durant le développement (partage de code, commits réguliers, etc.). Cela sera évalué en regardant la liste et les dates des commits.

Il est possible de pousser plusieurs versions dans le dépôt Git. Seule la **dernière version** présente à la date limite sera prise en compte.

9. Triche

Attention : il est fortement recommandé de résister à la tentation d'intégrer du code que vous n'avez pas vous-même développé, qu'il provienne d'autres groupes ou de recherches sur internet. Toute tentative de tricherie sera sanctionnée par une note de **0,5/20**. Avec les outils appropriés, il est relativement facile de détecter ce type de fraude (y compris si vous renommez les variables, les fichiers, ou modifiez légèrement la mise en forme).

Un dépôt contenant uniquement un ou deux commits très proches de la date limite, avec un projet complet arrivé « d'un seul coup », sera considéré comme **non conforme à l'esprit du projet** et pourra être pénalisé.

Vous pouvez éventuellement utiliser des ressources externes **de manière ponctuelle** (lecture de documentation, inspiration de petits extraits).

En cas de doute sur l'authenticité ou la compréhension du code, une courte **interrogation orale** pourra être organisée. Il est donc **préférable d'accepter une légère perte de points** en cas de fonctionnalité manquante plutôt que de risquer une note de **0,5/20**.